SPARC: Semantic Pareto-Aware Response Cache Optimization for Large Language Models

Gowsi S M
dept. of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai, India
gowsisubbian@gmail.com

Ajay S
dept. of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai, India
ajaysubramani.career@gmail.com

Aakash S
dept. of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai, India
aakashsrrm@gmail.com

Jerome Jestin A
dept. of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai, India
jeromejestina@gmail.com

M Gowthami
dept. of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai, India
gowthamimailme@gmail.com

***Abstract*—Semantic caching can reduce repeated large language model (LLM) inference by serving a stored response when a new query is sufficiently similar to a previous request. However, cache policies based mainly on similarity or a single ranking criterion may retain frequently reused responses even when the information associated with those responses changes over time. SPARC addresses this issue through Pareto-based cache management that considers cache utility and query volatility during admission and eviction. The implementation uses a token-saving proxy and a volatility score, compares candidates using Pareto dominance, and uses hypervolume contribution for pruning. An adaptive similarity threshold is also applied according to query characteristics. Experiments use the MOSS conversational dataset with real Sentence-Transformer embeddings and compare GPTCache, SCALM-style ranking, and SPARC across multiple workload scales. In the largest 2,000-query evaluation, SPARC records a 23.5 percent hit ratio and a 6.9 percent token saving ratio, while SCALM records 21.1 percent and 7.3 percent, respectively. The corresponding staleness rates are 1.3 percent and 2.0 percent, respectively. The results show that SPARC provides a volatility-aware alternative to scalar cache admission.**

Keywords— semantic caching, large language models, Pareto optimization, multi-objective optimization, token saving, query volatility, cache admission, cache eviction

I. INTRODUCTION

Large language model inference services repeatedly receive requests that are identical or semantically similar to previously processed queries. Recomputing responses for such requests introduces unnecessary inference computation, latency, and API cost. Semantic caching addresses this problem by storing generated responses and reusing them when a new request is sufficiently similar to a cached query.

GPTCache introduced an open-source semantic caching framework based on semantic similarity and vector retrieval [1]. SCALM subsequently studied semantic caching through semantic clustering and token-saving-ratio (TSR) based cache management, using workload reuse and response characteristics to estimate cache value [2]. These approaches establish cache reuse as an effective mechanism for reducing repeated inference, but a scalar utility criterion does not explicitly represent freshness risk.

A frequently reused query may concern information that changes rapidly, such as current events, weather, financial information, policies, or other temporal content. A cache entry can therefore have high reuse value while also carrying a higher probability of becoming stale. Treating utility and freshness as a single ranking objective can obscure this tradeoff. SPARC addresses the problem by formulating cache admission as a multi-objective decision.

The proposed framework has two current objectives: maximize a token-saving proxy and minimize query volatility. Rather than forcing the two quantities into one manually weighted score, Pareto dominance retains non-dominated candidates representing different utility–freshness trade-offs. When the cache is full, hypervolume contribution is used to select an entry for removal. The framework also uses an adaptive similarity threshold that changes with query volatility and domain-specific adjustments. The contributions of this work are fourfold:

- A two-objective Pareto admission and eviction mechanism for semantic response caching
- Explicit incorporation of query volatility as a freshness-related cache objective
- An adaptive similarity threshold for volatility- and domain-aware semantic matching
- A real-data evaluation of GPTCache, SCALM-style admission, and SPARC across multiple workload scales.

II. RELATED WORK

A. Semantic Caching

Semantic caching reduces repeated LLM inference by identifying semantically equivalent or sufficiently similar requests and reusing previously generated responses. GPTCache provides a general semantic cache using embeddings and similarity-based retrieval [1]. SCALM extends this direction with semantic clustering and token-saving-ratio based cache management [2]. MeanCache studies user-centric reuse patterns for LLM web services [3], while Cache Me If You Can investigates cost-aware reduction of LLM calls using an online teacher–student framework [4].

Freshness and context have also become relevant to LLM cache management. DKC-LLM investigates dynamic knowledge caching and semantic drift [5]. Context-based semantic caching incorporates contextual information into cache decisions [6], while freshness-aware semantic caching considers explicit freshness constraints for retrieval-augmented LLM systems [7]. These studies motivate treating

response reuse and information freshness as related but distinct concerns.

B. Cache Optimization and Multi-Objective Methods

Traditional cache replacement has used popularity, access frequency, and object cost to manage limited capacity [8]. Large-model inference has also been studied from a caching and multiplexing perspective [9], while DeepCache examines reuse for deep-learning inference [10]. Pareto optimization provides a general way to preserve non-dominated alternatives under competing objectives. NSGA-II is a standard multi-objective optimization method [11], and the Skyline operator provides a database formulation for non-dominated points [12]. Recent work has considered multi-objective KV-cache configuration and semantic or safe KV-cache management [13],[15]. In contrast, SPARC operates at the semantic response-cache layer rather than the internal model KV-cache layer.

III. Research Gap and Motivation

Existing semantic caching systems commonly emphasize semantic similarity, popularity, cost reduction, or freshness as separate mechanisms. GPTCache primarily focuses on similarity-based reuse, while SCALM uses a TSR-oriented ranking strategy [1],[2]. A response can nevertheless be simultaneously valuable for reuse and vulnerable to becoming stale. This motivates an explicit second objective for freshness risk. The project first examined this relationship using a rank–volatility audit on clustered MOSS data. For each cluster, TSR based priority and a volatility value were computed, and their monotonic association was measured using Spearman's rank correlation:

$$\rho = \mathrm{corr}_{\mathrm{Spearman}}(\mathrm{TSR\ Rank}, \mathrm{Volatility}) \quad (1)$$

The audit covered 261 clusters and obtained $\rho = +0.283$. Under the statistical criterion used in the analysis, $|\rho| > 0.122$ at $\alpha = 0.05$. The positive association indicates that higher TSR priority and higher volatility can occur together in the analyzed workload. This does not establish causality, but it supports treating volatility as a distinct cache-management signal. SPARC therefore formulates admission as:

$$\max \mathrm{TokenSavingProxy}(c) \quad (2)$$

$$\min \mathrm{Volatility}(c) \quad (3)$$

This formulation preserves different non-dominated tradeoffs instead of imposing a fixed scalar weight between the two properties.

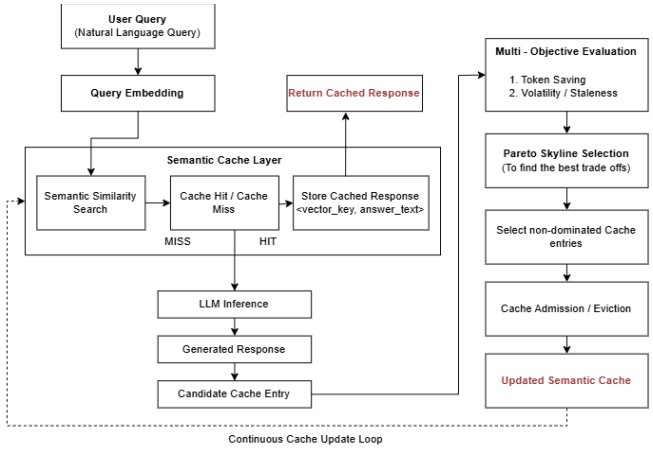


Fig. 1. SPARC semantic response-cache architecture and admission pipeline.

IV. SPARC Framework

A. System Overview

The semantic cache pipeline embeds each incoming query and performs a vector similarity search against stored queries. If a suitable match is found, the associated response is returned. Otherwise, the query is sent to the LLM and the resulting response becomes a candidate cache entry. The candidate is evaluated using a token-saving proxy and a volatility score. A dominated candidate is rejected; a non-dominated candidate can be admitted, after which capacity-based pruning uses hypervolume contribution. Fig. 1 summarizes the implemented pipeline.

B. Token-Saving Objective

For a cache entry e, the current admission implementation uses a token-saving proxy derived from query and response token counts:

$$S(e) = T_{query}(e) + T_{response}(e) \quad (4)$$

Because the implementation uses minimizing objective values, the first Pareto objective is

$$o_1(e) = -S(e) \quad (5)$$

A lower o1 therefore corresponds to a larger token-saving proxy. This admission proxy is distinct from the workload-level Token Saving Ratio reported during evaluation.

C. Volatility Objective

The second objective represents freshness risk. The current implementation uses a lightweight rule-based classifier with three categories:

- **STABLE** – Information changes slowly over time.
- **TEMPORAL** – Information changes with time or events.
- **PERSONAL** – Information depends on user-specific context.

Each category is assigned a bounded numerical volatility value, with lower values representing lower freshness risk. Temporal volatility represents the likelihood that the information associated with a query may change over time. A query may remain semantically similar to a previous request while the information required to answer it has changed. Therefore, semantic similarity alone does not guarantee that a cached response remains appropriate. SPARC treats volatility as a separate objective so that responses associated with frequently changing information can be distinguished from more stable cache entries. The objective is

$$o_2(e) = Volatility(e) \quad (6)$$

D. Pareto Admission and Eviction

For two entries A and B, A dominates B if it is no worse on every objective and strictly better on at least one:

$$A \succ B \Leftrightarrow [\forall i,\ o_i(A) \le o_i(B)] \wedge \left[\exists j,\ o_j(A) < o_j(B)\right] \quad (7)$$

The Pareto frontier is the set of entries for which no other cache entry dominates the candidate. This avoids reducing token utility and volatility to a single manually weighted score.

When a new cache candidate is generated, SPARC evaluates it against the entries currently stored in the cache

using the two objectives of token-saving utility and query volatility. If an existing entry dominates the candidate, the candidate is rejected because another cache entry provides no worse objective values and is strictly better in at least one objective. If the candidate is non-dominated, it is considered for admission.

When sufficient cache capacity is available, the non-dominated candidate is inserted directly. If the cache exceeds its capacity after admission, SPARC uses hypervolume contribution to identify an entry for eviction **[17]**. Entries contributing less to the coverage of the two-dimensional objective space receive higher eviction priority. This allows the cache to preserve diverse utility-volatility trade-offs rather than relying on a single scalar ranking.

E. SPARC Cache Management Algorithm

The complete cache-management procedure is summarized in Algorithm 1.

Input: Query q, cache C, capacity K

1. Generate the semantic embedding of q.
2. Search C for a semantically similar cached entry.
3. Apply the adaptive similarity threshold $\tau(q)$.
4. If a valid cached match is found:
5. Return the associated response.
6. Otherwise, send q to the LLM and obtain response r.
7. Create cache candidate e = (q, r).
8. Compute $o_1(e)$ and $o_2(e)$.
9. Compare e with existing entries using Pareto dominance.
10. If e is dominated:
11. Reject e.
12. Otherwise:
13. Admit e into C.
14. If |C| > K:
15. Compute the hypervolume contribution of cache entries.
16. Evict the entry with the lowest contribution.
17. Return the updated cache C.

Algorithm 1: SPARC Cache Management

F. Adaptive Similarity Thresholds

The matching threshold is adjusted according to query characteristics. The implemented threshold is

$$\tau(q) = \text{clip}(\tau_{base} + \Delta_{domain} - \beta Volatility(q), 0,1)) \quad (8)$$

The base threshold used in the reported experiments is approximately 0.90. Domain adjustments are available for medical, code, legal, and general queries. The resulting value is clipped to [0, 1].

V. EXPERIMENTAL METHODOLOGY

A. Dataset and Implementation

The primary evaluation uses the MOSS conversational dataset [16]. Real semantic experiments use the sentence-transformers/all-MiniLM-L6-v2 model [18], which produces 384-dimensional embeddings. FAISS is supported as the vector-search layer [19], with an in-memory implementation used for controlled testing. Semantic clustering uses DBSCAN with $\epsilon = 0.30$ and a minimum cluster size of five.

Three systems are evaluated: GPTCache with unconditional admission, a SCALM-style TSR-ranked cache, and SPARC with token-saving proxy plus volatility. A mock embedding provider is used only for smoke tests and development verification; the main reported real-data results use sentence-transformer embeddings.

TABLE I
REAL-EMBEDDING VALIDATION CONFIGURATIONS

Queries	Capacity	Warmup	Threshold	Embedding
300	30	50	0.90	MiniLM
600	40	100	0.90	MiniLM
2000	100	100	0.90	MiniLM

B. Validation Protocol

The validation process was deliberately staged. A 100- query mock-embedding smoke test was used to verify end-to-end execution. A 300-query real-embedding run used cache capacity 30 and warmup count 50. A 600-query paper-matched run used capacity 40 and warmup count 100. The larger validation used 2,000 queries, cache capacity 100, warmup count 100, a 0.90 similarity threshold, and real embeddings. The latter runs provide the main evidence reported below.

C. Metrics

The evaluation reports cache hit ratio, Token Saving Ratio (TSR), the project's volatility-based staleness rate, false-hit rate, and admission overhead. Hit ratio measures the percentage of queries served from cache. TSR measures the proportion of response tokens avoided through reuse relative to the corresponding no-cache workload. Staleness is the project's proxy for hits associated with volatile responses. False-hit rate is based on the implemented similarity validation procedure and is not treated as a definitive semantic-correctness benchmark. Admission overhead measures additional computation associated with candidate evaluation and Pareto processing.

VI. RESULTS

A. Multi-Scale SCALM and SPARC Validation

Table II summarizes the reported real-embedding validation runs. SPARC recorded 0.0% observed staleness at 300 and 600 queries and 1.3% at 2,000 queries, while SCALM recorded 6.3%, 5.2%, and 2.0%, respectively. The hit-ratio relationship varied with workload size: SPARC recorded 8.0% versus 6.4% for SCALM at 300 queries, 8.6% versus 11.6% at 600 queries, and 23.5% versus 21.1% at 2,000 queries.

The 2,000-query run is the largest reported validation and provides the main head-to-head comparison. At this scale, SPARC records a 23.5% hit ratio compared with 21.1% for SCALM, a difference of 2.4 percentage points (approximately 11.4% relative to the SCALM value). Its observed staleness rate is 1.3%, compared with 2.0% for SCALM, corresponding to a 35% relative reduction. These results are descriptive measurements for the reported workload and do not establish universal performance across other datasets.

TABLE II
MULTI-SCALE REAL-EMBEDDING VALIDATION OF SCALM AND SPARC

Queries	SCALM Hit	SPARC Hit	SCALM Stale	SPARC Stale
300	6.4%	8.0%	6.3%	0.0%
600	11.6%	8.6%	5.2%	0.0%
2000	21.1%	23.5%	2.0%	1.3%

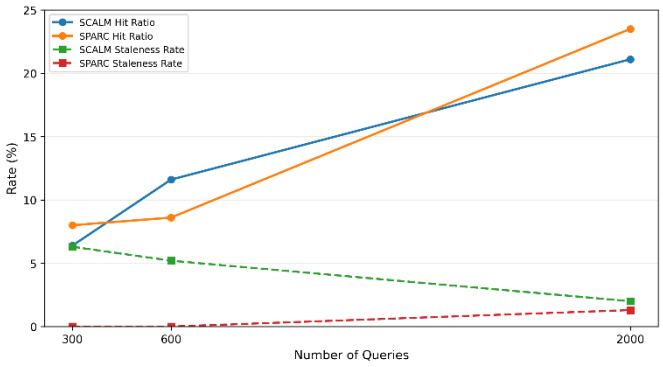


Fig. 2. Multi-scale comparison of hit ratio and staleness rate for SCALM and SPARC.

B. *Multi-Scale Real-Embedding Validation Of SCALM And SPARC*

Table III presents the detailed 2,000-query comparison. SPARC records a 23.5% hit ratio and a 6.9% Token Saving Ratio, while SCALM records 21.1% and 7.3%, respectively. Both systems record a 0.0% false-hit rate, and their staleness rates are 1.3% for SPARC and 2.0% for SCALM. The measured additional computation for Pareto processing is approximately 4–5ms per candidate in the experimental environment.

TABLE III
2,000-QUERY HEAD-TO-HEAD VALIDATION

Metric	SCALM	SPARC
Hit Ratio	21.1%	23.5%
Token Saving Ratio	7.3%	6.9%
Staleness Rate	2.0%	1.3%
False-Hit Rate	0.0%	0.0%

C. *Token Savings, False Hits, and Overhead*

At 2,000 queries, the reported Token Saving Ratio is 7.3% for SCALM and 6.9% for SPARC. Thus, the volatility-aware policy retains a similar magnitude of token saving performance while introducing the additional volatility objective. The reported false-hit rate for both systems in the 2,000-query head-to-head comparison is 0.0%. The validation notes report approximately 4–5 ms of additional computation per candidate for Pareto processing. This figure should be interpreted as an implementation-level overhead measurement under the project's current environment rather than a general latency guarantee.

D. *GPTCache Baseline*

The project's earlier MOSS evaluation also reports GPTCache at 31.8% hit ratio, 8.6% Token Saving Ratio, 0.8% staleness, and 0.0% false-hit rate under its implemented baseline configuration. GPTCache uses unconditional admission, so these values are not interpreted as a direct substitute for the staged SCALM–SPARC validation above. They provide a baseline reference for the behavior of a cache without the proposed Pareto admission filter.

VII. DISCUSSION

The multi-scale results show that the effect of Pareto admission depends on workload scale and cache state. The hit-ratio difference changes direction between the 300-, 600-, and 2,000-query runs, so the current evidence should not be summarized as a monotonic hit-rate advantage. The consistent observation across the three reported scales is the lower measured staleness for SPARC than SCALM. The central design difference is the objective representation.

A. *Pareto Representation Versus Scalar Admission*

SCALM primarily orders candidates according to a scalar token-saving criterion, whereas SPARC evaluates the pair

$$\mathbf{o}(e) = \left(-TokenSavingProxy(e), Volatility(e)\right) \quad (9)$$

A high-utility but volatile entry can therefore coexist with a lower-utility stable entry when neither dominates the other. This preserves multiple operating points without manually selecting a fixed trade-off weight.

B. *Rank--Volatility Conflict*

The rank–volatility audit provides additional motivation. The positive correlation observed across 261 clusters indicates that the analyzed workload can contain clusters that are both highly prioritized by TSR and comparatively volatile. The correlation is workload-specific and does not by itself demonstrate causality or generalization.

C. *Context-Dependent Cache Invalidation*

Semantic similarity may not capture all contextual differences between queries. For example, programming and personalized requests may require different responses despite high semantic similarity. The current SPARC implementation primarily uses semantic query representation, so incorporating session or user context could improve cache-key precision.

D. *Limitations*

The current implementation also has several limitations. First, the token-saving admission objective is a proxy based on entry token counts rather than a direct prediction of future workload-level savings. Second, the volatility classifier is rule based and uses design-assigned categories rather than human annotated freshness labels. Third, staleness and false-hit rates are proxy measurements rather than comprehensive semantic correctness evaluations. Fourth, the experiments focus on MOSS, so domain-specific workloads are needed to assess generalization. Finally, the reported overhead was measured in the project's environment and requires controlled latency experiments before broader performance conclusions can be made.

E. *Computational Complexity and Scalability*

The Pareto-based admission process introduces additional computation compared with scalar cache ranking because each candidate may need to be compared with existing cache entries across multiple objectives. With a straightforward implementation, Pareto comparison over $n$cache entries has a worst-case complexity of $O(n^2)$. The current implementation

uses two objectives, which keeps the objective space small and makes hypervolume-based pruning practical for the cache capacities evaluated in this work.

The additional computation is incurred primarily during cache admission rather than during a direct cache hit. Therefore, the practical impact depends on the frequency of candidate insertions and the cache capacity. The reported admission overhead was measured in the experimental environment, while broader scalability evaluation across larger cache sizes and higher request rates remains necessary.

F. Future Work

Future work will focus on improving both the objectives and the evaluation of SPARC. The token-saving objective can be extended from an entry-level proxy to a workload-aware estimate of expected future savings. The rule-based volatility classifier can also be replaced or supplemented with stronger freshness estimation using temporal and contextual signals.

Further evaluation can incorporate additional objectives such as measured latency, semantic response quality, and context consistency. Experiments on larger and domain-specific datasets, including workloads with stronger temporal and personalized characteristics, will also be useful for evaluating the generalization and scalability of the framework. These extensions can provide a broader assessment of the utility-freshness trade-off in semantic response caching.

VIII. CONCLUSION

This paper presented SPARC, a Pareto-based semantic response-cache management framework that jointly considers cache utility and query volatility. The implementation uses a token-saving proxy and a rule-based volatility objective for admission, Pareto dominance for candidate selection, hypervolume contribution for capacity-based pruning, and an adaptive similarity threshold for cache lookup.

On the reported real-embedding MOSS validation, SPARC reduced the project's measured staleness relative to SCALM at 300, 600, and 2,000 queries, with staleness values of 0.0%, 0.0%, and 1.3%, respectively, versus 6.3%, 5.2%, and 2.0% for SCALM. At 2,000 queries, the system achieved a 23.5% hit ratio and 6.9% Token Saving Ratio, compared with 21.1% and 7.3% for SCALM. The results support the feasibility of incorporating volatility as a separate cache admission objective, while also showing that further evaluation is needed for semantic correctness, latency, broader datasets, and workload-aware utility estimation.

ACKNOWLEDGMENT

The authors thank M. Gowthami for guidance and Easwari Engineering College for providing the academic environment and computational support required for this work. This work was conducted as part of the Final Year Project program in the Department of Computer Science and Engineering.

REFERENCES

[1] B. Fu and D. Feng, "GPTCache: An Open-Source Semantic Cache for LLM Applications," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, pp. 212–218, 2023.

[2] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu, "SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models," in *Proceedings of the IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, pp. 1–10, 2024, doi: 10.1109/IWQoS61813.2024.10682957.

[3] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-Centric Semantic Caching for LLM Web Services," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025.

[4] I. Stogiannidis, S. Vassos, P. Malakasiotis, and I. Androutsopoulos, "Cache me if you Can: an Online Cost-aware Teacher-Student Framework to Reduce the Calls to Large Language Models," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 14999– 15008, 2023.

[5] A. Khaliq and K. J. Adebayo, "DKC-LLM: Dynamic Knowledge Caching for Large Language Models in Business Applications," *IEEE Access*, vol. 14, pp. 22318–22334, 2026, doi: 10.1109/ACCESS.2026.3662344.

[6] R. Mohandoss, "Context-based Semantic Caching for LLM Applications," in *2024 IEEE Conference on Artificial Intelligence (CAI)*, pp. 371–376, 2024, doi: 10.1109/CAI59869.2024.00075.

[7] M. Mansoor, T. Ahmad, and Y.-C. Yoon, "Risk-Constrained FreshnessAware Semantic Caching for Open-Web Retrieval-Augmented LLMs," arXiv preprint arXiv:2607.04281, 2026.

[8] S. Jin and A. Bestavros, "Popularity-Aware Greedy Dual-Size Web Proxy Caching Algorithms," in *Proceedings of the 20th International Conference on Distributed Computing Systems (ICDCS)*,pp. 254–261, 2000.

[9] B. Zhu, Y. Sheng, L. Zheng, C. W. Barrett, M. I. Jordan, and J. Jiao, "On Optimal Caching and Model Multiplexing for Large Model Inference," arXiv preprint arXiv:2306.02003, 2023.

[10] M. Xu, M. Zhu, Y. Liu, F. X. Lin, and X. Liu, "DeepCache: Principled Cache for Mobile Deep Vision," in *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2018, doi: 10.1145/3241539.3241563.

[11] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002, doi: 10.1109/4235.996017.

[12] S. Borzs ¨ onyi, D. Kossmann, and K. Stocker, "The Skyline Operator," in ¨ *Proceedings of the 17th International Conference on Data Engineering (ICDE)*, pp. 421–430, 2001.

[13] X. Zheng et al., "Adaptive Multi-Objective Tiered Storage Configuration for KV Cache in LLM Service," arXiv preprint arXiv:2603.08739, 2026.

[14] A. Abaskohi, G. Carenini, P. West, and Y. He, "SeKV: ResolutionAdaptive KV Cache with Hierarchical Semantic Memory for LongContext LLM Inference," arXiv preprint arXiv:2606.31145, 2026.

[15] K. Chu, Z. Shen, D. Xiang, and W. Zhang, "SafeKV: Safe KV-Cache Sharing in LLM Serving," in *Proceedings of the Machine Learning for Computer Architecture and Systems (MLArchSys)*, 2025.

[16] T. Sun et al., "MOSS: An Open Conversational Large Language Model," *Machine Intelligence Research*, vol. 21, 2024.

[17] K. Van Moffaert, M. M. Drugan, and A. Nowe, "Hypervolume- ´ based Multi-Objective Reinforcement Learning," in *Evolutionary MultiCriterion Optimization*, Lecture Notes in Computer Science, vol. 7811, pp. 352–366, Springer, 2013, doi: 10.1007/978-3-642-37140-0 28.

[18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLPIJCNLP)*, pp. 3982–3992, 2019, doi: 10.18653/v1/D19-1410.

[19] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazare, M. Lomeli, L. Hosseini, and H. Jegou, "The Faiss Library," *IEEE Transactions on Big Data*, vol. 12, no. 2, pp. 346–367, Apr. 2026, doi: 10.1109/TBDATA.2025.3618474.

[20] G. Liu, Y. Liu, J. Wang, H. Du, D. Niyato, J. Kang, and Z. Xiong, "Adaptive Contextual Caching for Mobile-Edge Large Language Model Service: A Proposed Framework," *IEEE Vehicular Technology Magazine*, vol. 21, no. 2, pp. 120–130, 2026, doi: 10.1109/MVT.2025.3608414